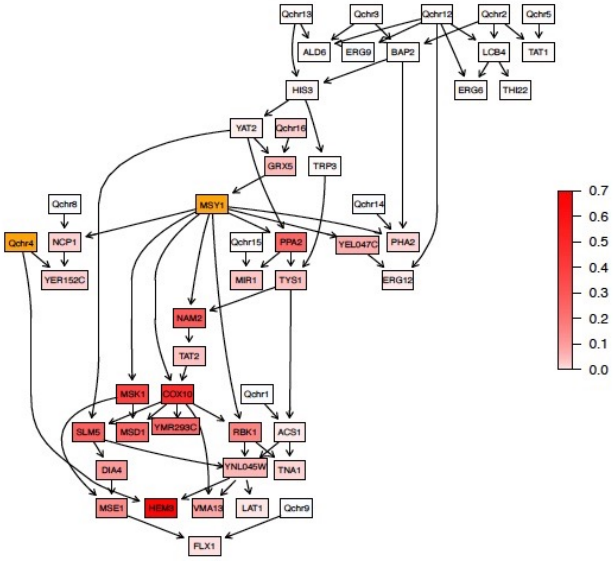
Probabilistic Reasoning



Rachael Hageman Blair

Statistical Data Mining 2

** Some Slides adopted from Marco Scutari – Short Course in Bayesian Networks

Evidence and Queries

Probabilistic reasoning on BNs works in the framework of Bayesian statistics and focuses on the computation of posterior probabilities or densities.

For example, suppose we have learned a BN $\mathcal{B}$ with DAG G and parameters Θ. We want to use $\mathcal{B}$ to investigate the effects of a new piece of evidence $\mathbf{E}$ using the knowledge encoded in $\mathcal{B}$, that is, to investigate the posterior distribution

$$\mathrm{P}(\mathbf{X} \mid \mathbf{E}, \mathcal{B}) = \mathrm{P}(\mathbf{X} \mid \mathbf{E}, G, \Theta).$$

Questions that can be asked are called queries and are typically an event of interest. The two most common queries are conditional probability (CPQ) and maximum a posteriori (MAP) queries, also known as most probable explanation (MPE) queries.

Evidence and Queries

- Hard evidence: an instantiation of one or more variables in the BN. In other words,

$$\mathbf{E} = \{X_{i_1} = e_1, X_{i_2} = e_2, \ldots, X_{i_k} = e_k\},$$

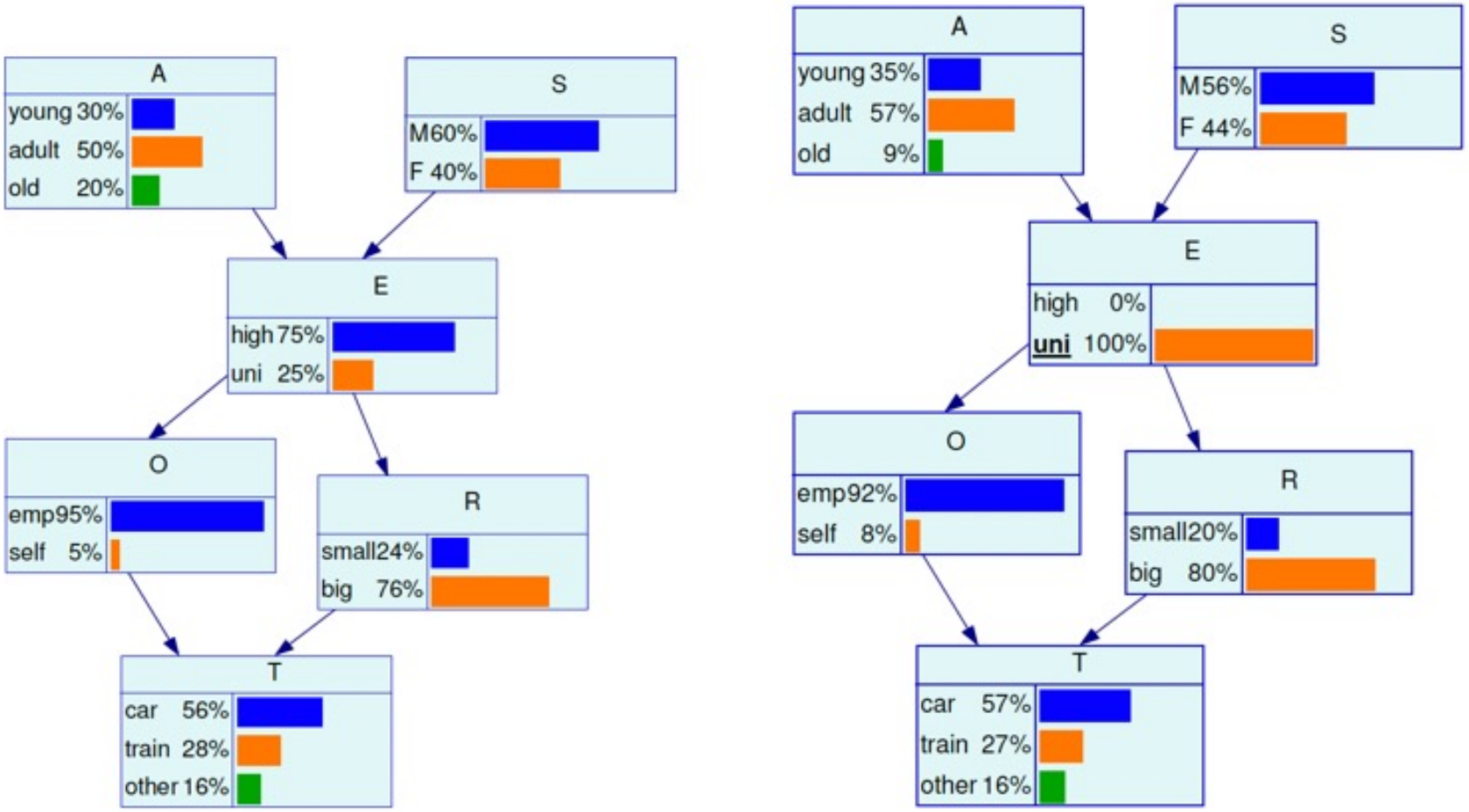
which ranges from the value of a single variable X_i to a complete specification for $\mathbf{X}$ (such a new partial or complete observation).

- Soft evidence: a new distribution for one or more variables in the network. Since both the network structure and the distributional assumptions are treated as fixed, soft evidence is usually specified as a new set of parameters,

$$\mathbf{E} = \left\{X_{i_1} \sim (\Theta_{X_{i_1}}), X_{i_2} \sim (\Theta_{X_{i_2}}), \ldots, X_{i_k} \sim (\Theta_{X_{i_k}})\right\}.$$

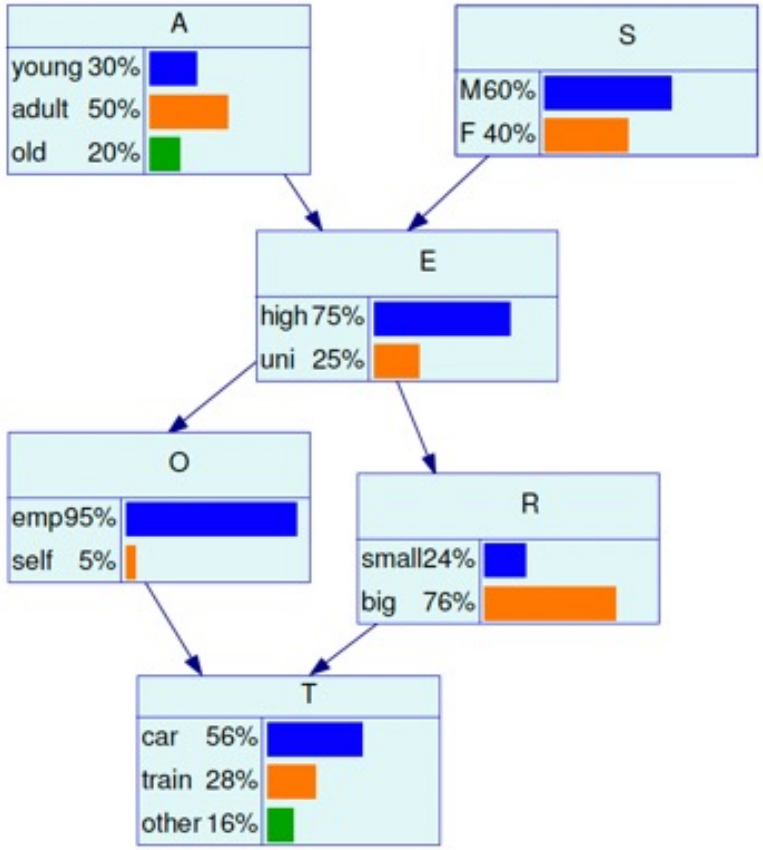
This new distribution may be, for instance, the null distribution in a hypothesis testing problem.

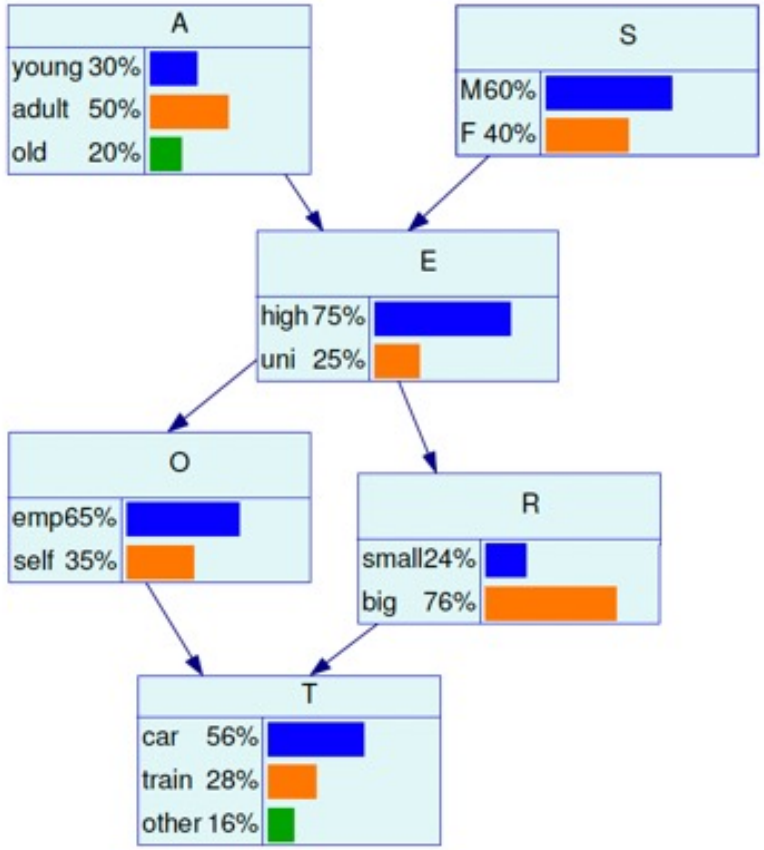
Evidence and Queries – Hard Evidence



The original survey BN (left), and the posterior BN with hard evidence on Education (right).

Evidence and Queries – Soft Evidence





The original survey BN (left), and the posterior BN with soft evidence on Employment (right).

Conditional Probability Queries

Conditional probability queries are concerned with

$$\mathrm{CPQ}(\mathbf{Q} \mid \mathbf{E}, \mathcal{B}) = \mathrm{P}(\mathbf{Q} \mid \mathbf{E}, G, \Theta) = \mathrm{P}(X_{j_1}, \ldots, X_{j_l} \mid \mathbf{E}, G, \Theta),$$

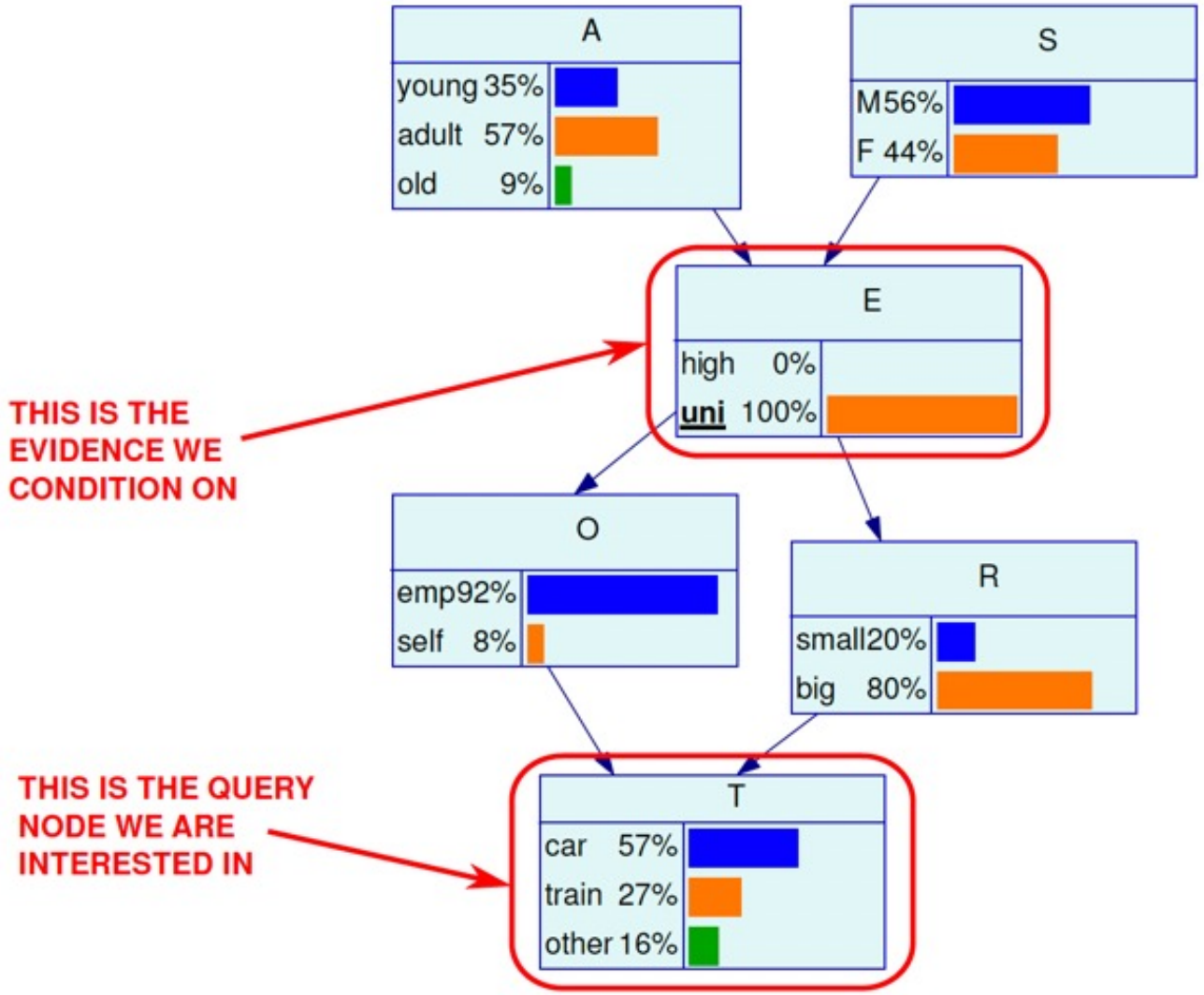
for some query variables $\mathbf{Q}$ given some hard evidence $\mathbf{E}$ on other variables, that is, the marginal posterior probability distribution of $\mathbf{Q}$,

$$\mathrm{P}(\mathbf{Q} \mid \mathbf{E}, G, \Theta) = \int \mathrm{P}(\mathbf{X} \mid \mathbf{E}, G, \Theta)\, d(\mathbf{X} \setminus \mathbf{Q}).$$

This class of queries has many useful applications due to their versatility.

For instance, we can assess the odds of an unfavourable outcome $\mathbf{Q}$ can for different sets of hard evidence $\mathbf{E}_1, \mathbf{E}_2, \ldots, \mathbf{E}_m$ of one or more related variables.

Conditional Probability Queries



Maximum a Posteriori Queries

Maximum a posteriori queries are concerned with finding the configuration $\mathbf{q}^*$ of $\mathbf{Q}$ that has the highest posterior probability (for discrete BNs) or the maximum posterior density (for GBNs and CLGBNs),

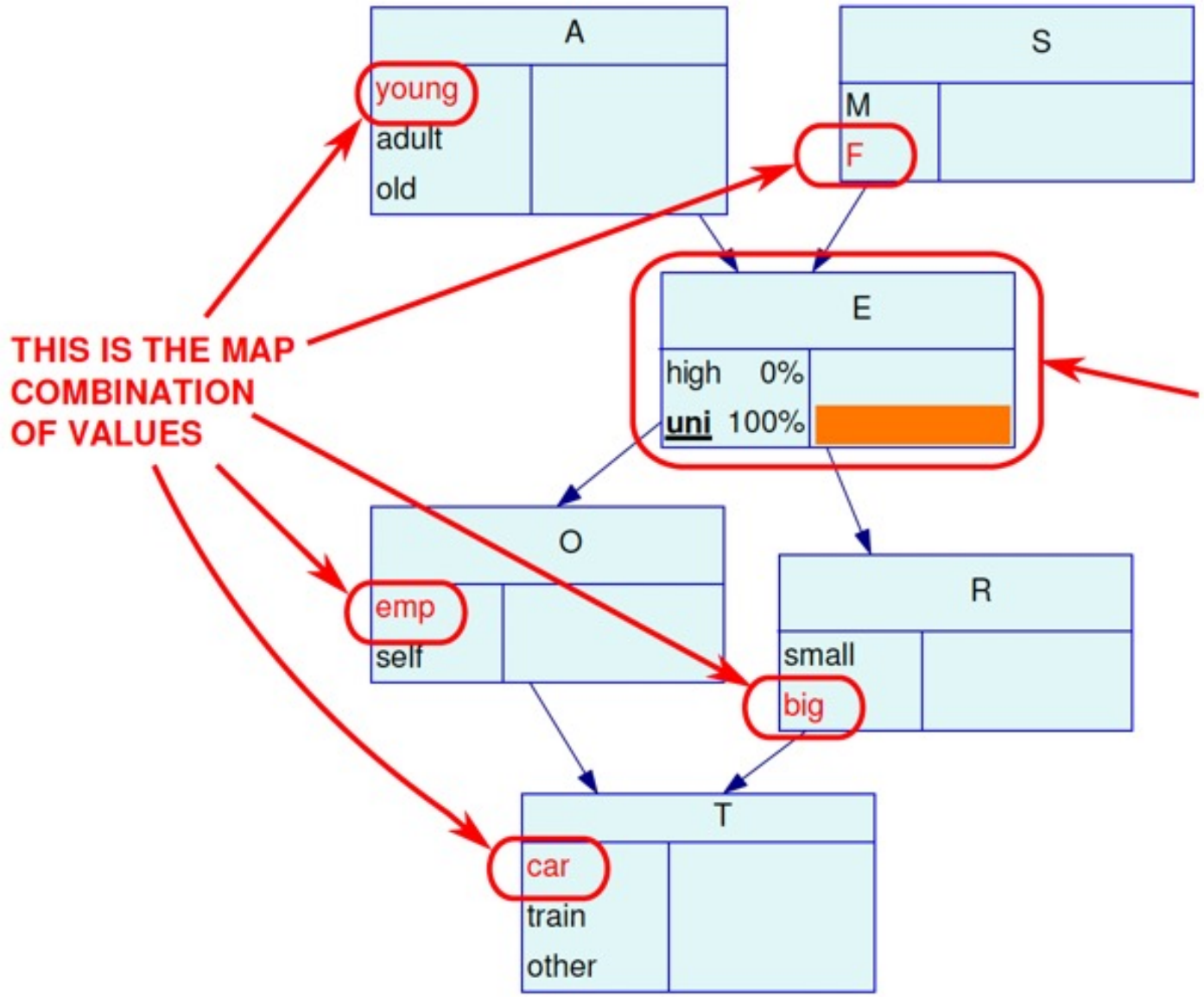
$$\text{MAP}(\mathbf{Q} \mid \mathbf{E}, \mathcal{B}) = \mathbf{q}^* = \underset{\mathbf{q}}{\text{argmax}}\, \text{P}(\mathbf{Q} = \mathbf{q} \mid \mathbf{E}, G, \Theta). \tag{4}$$

Two main applications:

- imputing missing data, where the variables in $\mathbf{Q}$ are not observed and are imputed from those in $\mathbf{E}$;
- comparing $\mathbf{q}^*$ with the observed values for the variables in $\mathbf{Q}$.

NOTE: $\mathbf{q}^*$ is not the collection of the values with the highest posterior in each posterior marginal distribution, those distributions are not independent!

Maximum a Posteriori Queries



Exact and Approximate Inference

Algorithms for belief updating can be classified either as

- Exact: algorithms that combine repeated applications of Bayes theorem with local computations to obtain the exact value of $\mathrm{P}(\mathbf{Q} \mid \mathbf{E}, G, \Theta)$. The two best known are
  - variable elimination; and
  - belief updates based on junction trees.
- Approximate: algorithms that use Monte Carlo simulations to sample from the global distribution and thus estimate $\mathrm{P}(\mathbf{Q} \mid \mathbf{E}, G, \Theta)$. In computer science, these random samples are often called particles, and the algorithms that make use of them are known as particle filters. The two best known are
  - logic sampling; and
  - likelihood weighting.

Approximate algorithms tend to scale better to larger number of variables since they are usually embarrassingly parallel; exact algorithms tend to be more sequential and iterative in nature.

Junction Tree Formation

1. **Moralise:** create the moral graph of the BN $\mathcal{B}$.
2. **Triangulate:** break every cycle spanning 4 or more nodes into sub-cycles of exactly 3 nodes by adding arcs to the moral graph, thus obtaining a triangulated graph.
3. **Cliques:** identify the cliques $C_1, \ldots, C_k$ of the triangulated graph, i.e., maximal subsets of nodes in which each element is adjacent to all the others.
4. **Junction Tree:** create a tree in which each clique is a node, and adjacent cliques are linked by arcs. The tree must satisfy the running intersection property: if a node belongs to two cliques C_i and C_j, it must be also included in all the cliques in the (unique) path that connects C_i and C_j.
5. **Parameters:** use the parameters of the local distributions of $\mathcal{B}$ to compute the parameter sets of the compound nodes of the junction tree.

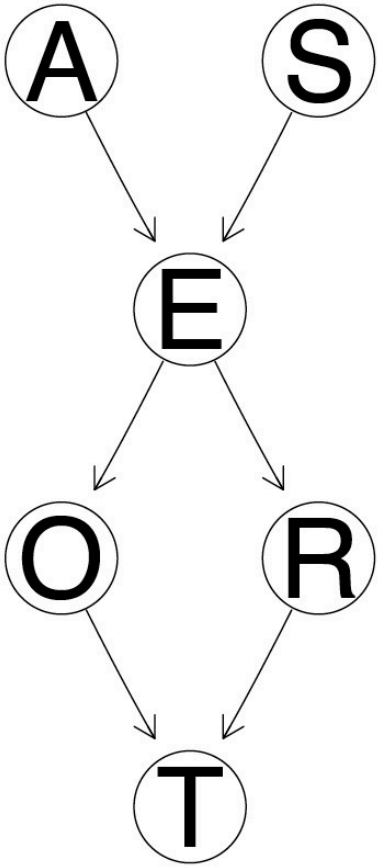
Back to our Train Example

Consider a simple, hypothetical survey whose aim is to investigate the usage patterns of different means of transport, with a focus on cars and trains.

- Age (A): *young* for individuals below 30 years old, *adult* for individuals between 30 and 60 years old, and *old* for people older than 60.
- Sex (S): *male* or *female*.
- Education (E): *up to high school* or *university degree*.
- Occupation (O): *employee* or *self-employed*.
- Residence (R): the size of the city the individual lives in, recorded as either *small* or *big*.
- Travel (T): the means of transport favoured by the individual, recorded either as *car*, *train* or *other*.

The nature of the variables recorded in the survey suggests how they may be related with each other.

Back to our Train Example



That is a prognostic view of the survey as a BN:

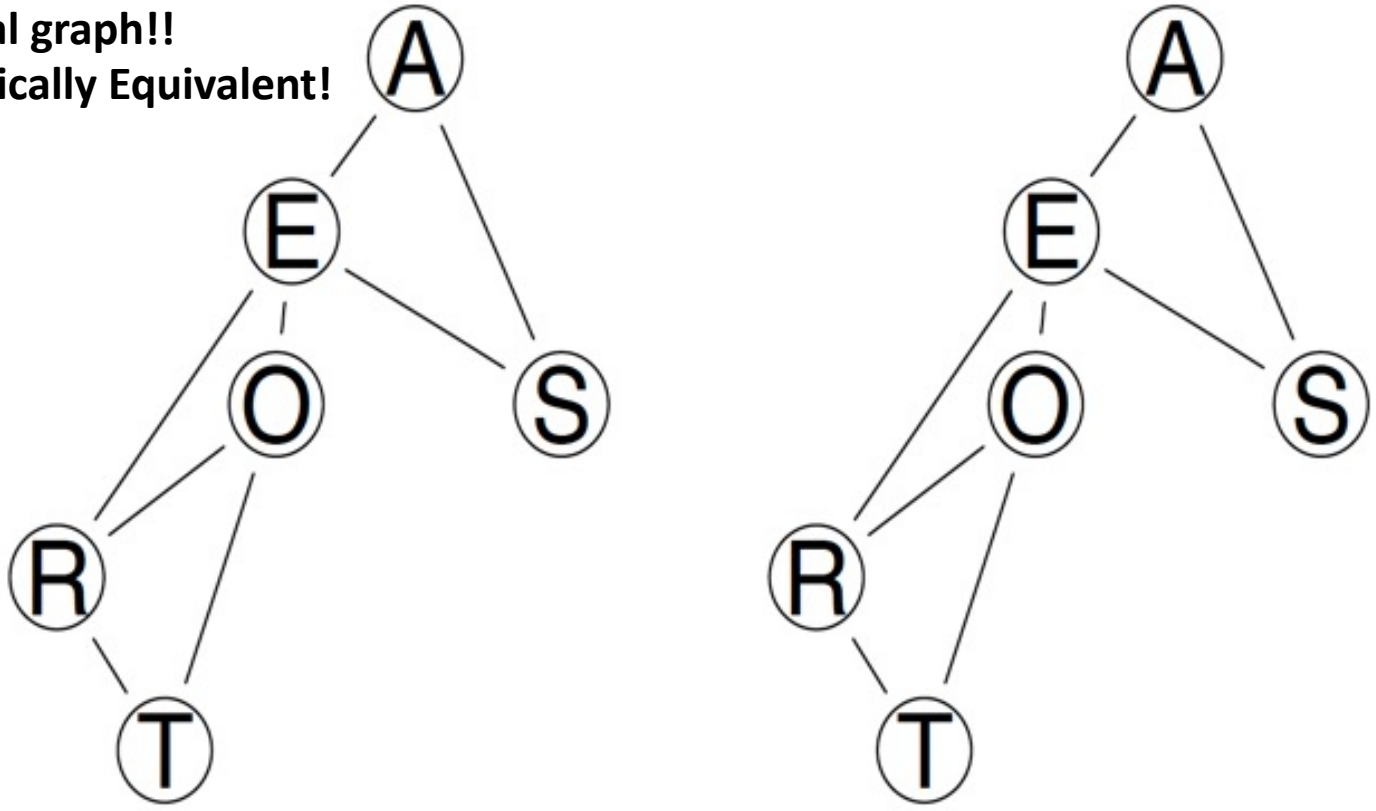
1. the blocks in the experimental design on top (e.g. stuff from the registry office);
2. the variables of interest in the middle (e.g. socio-economic indicators);
3. the object of the survey at the bottom (e.g. means of transport).

Variables that can be thought as "causes" are on above variables that can be considered their "effect", and confounders are on above everything else.

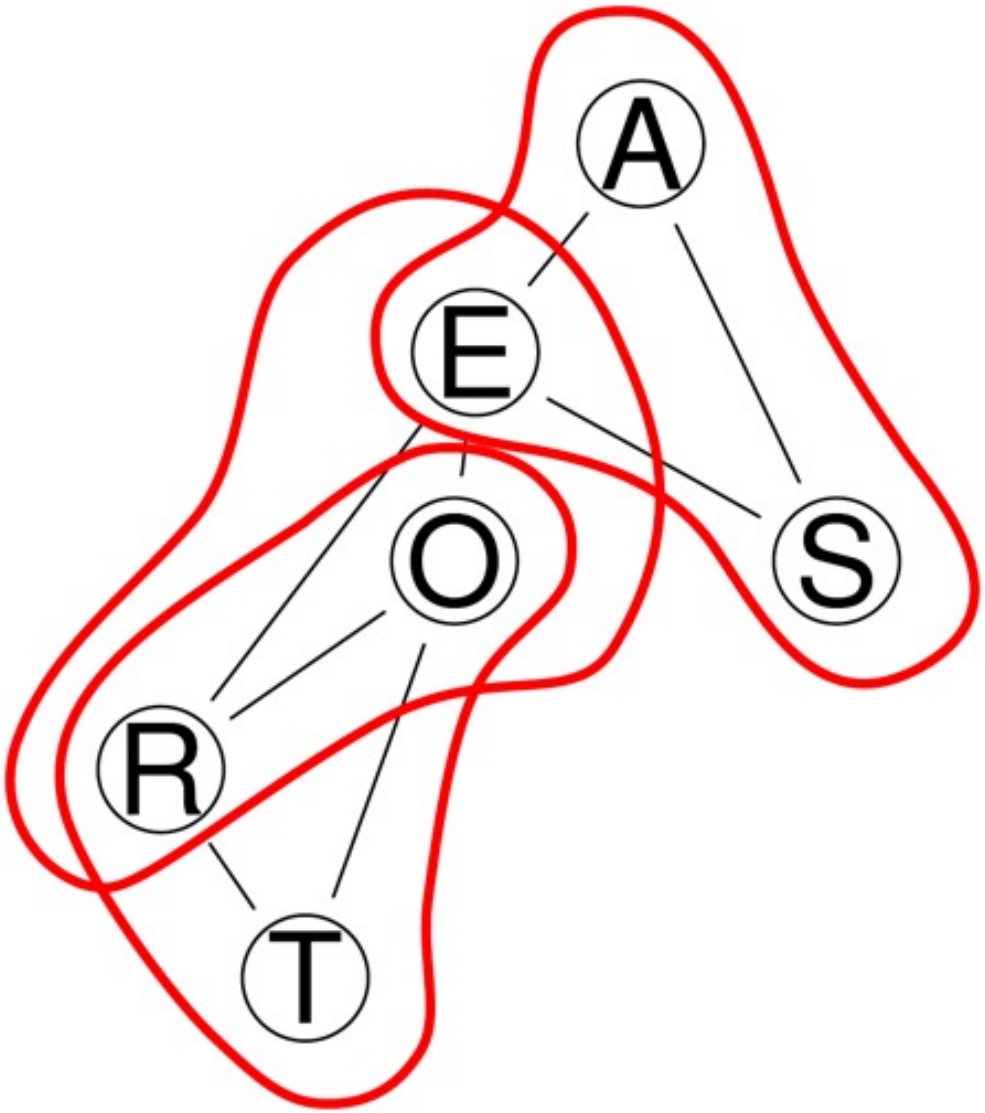
Back to our Train Example – moral graphs

```
survey.dag1 = model2network("[A][S][E|A:S][O|E][R|E][T|O:R]")
survey.dag2 = model2network("[A|E][S|A:E][E|O:R][O|R:T][R|T][T]")
par(mfrow = c(1, 2))
graphviz.plot(moral(survey.dag1))
graphviz.plot(moral(survey.dag2))
```

Same moral graph!!
Probabilistically Equivalent!



Back to our Train Example – cliques



The moral graph is already triangulated, and we can see three cliques:

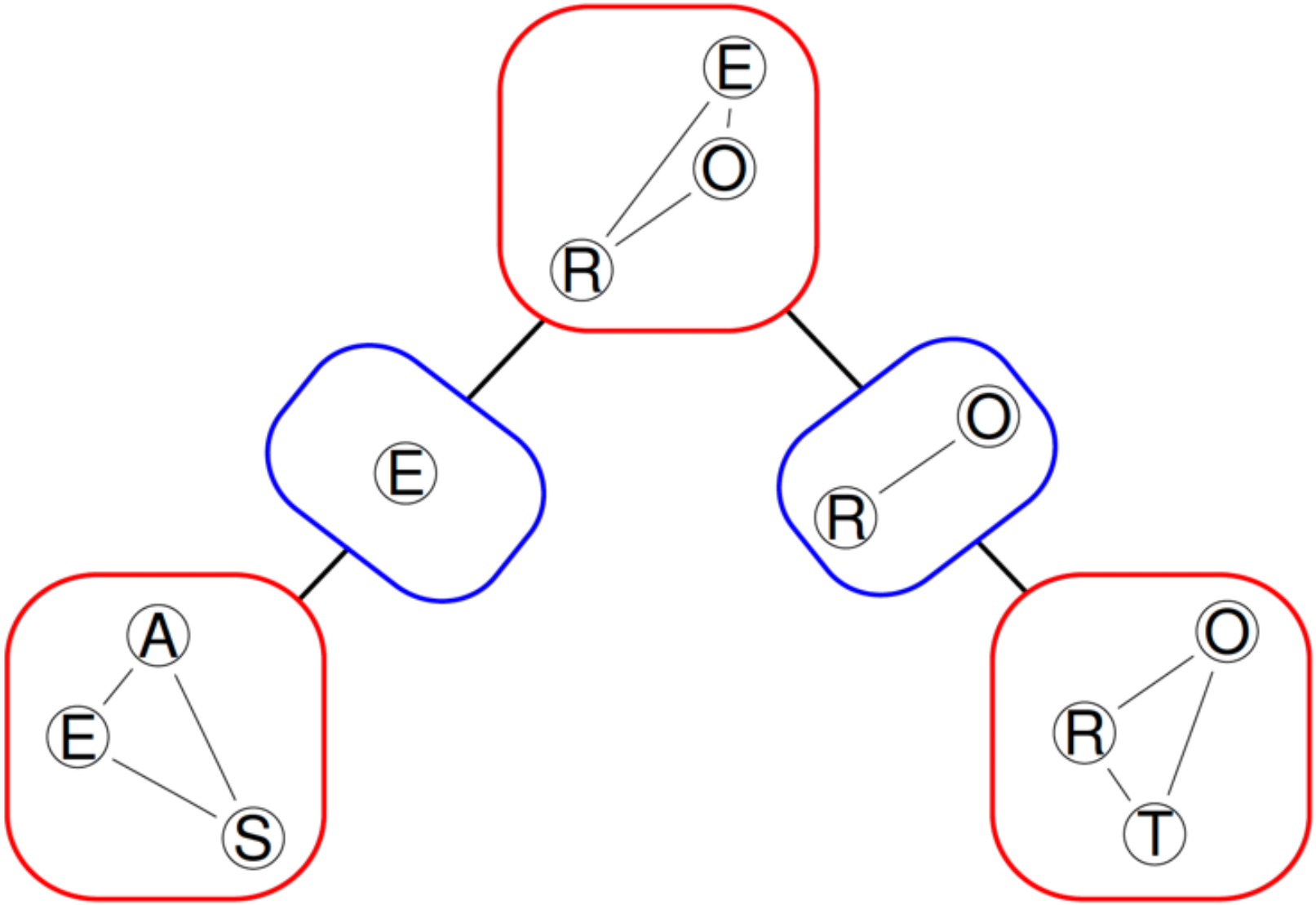
$$C_1 = \{A, E, S\}$$
$$C_2 = \{E, O, R\}$$
$$C_3 = \{O, R, T\}$$

with separators:

$$S_{12} = \{E\}$$
$$S_{23} = \{O, R\}$$

which we can use to build the junction tree.

Back to our Train Example – cliques



Back to our Train Example – assignments

In this example on the survey BN, the parameters for the cliques are:

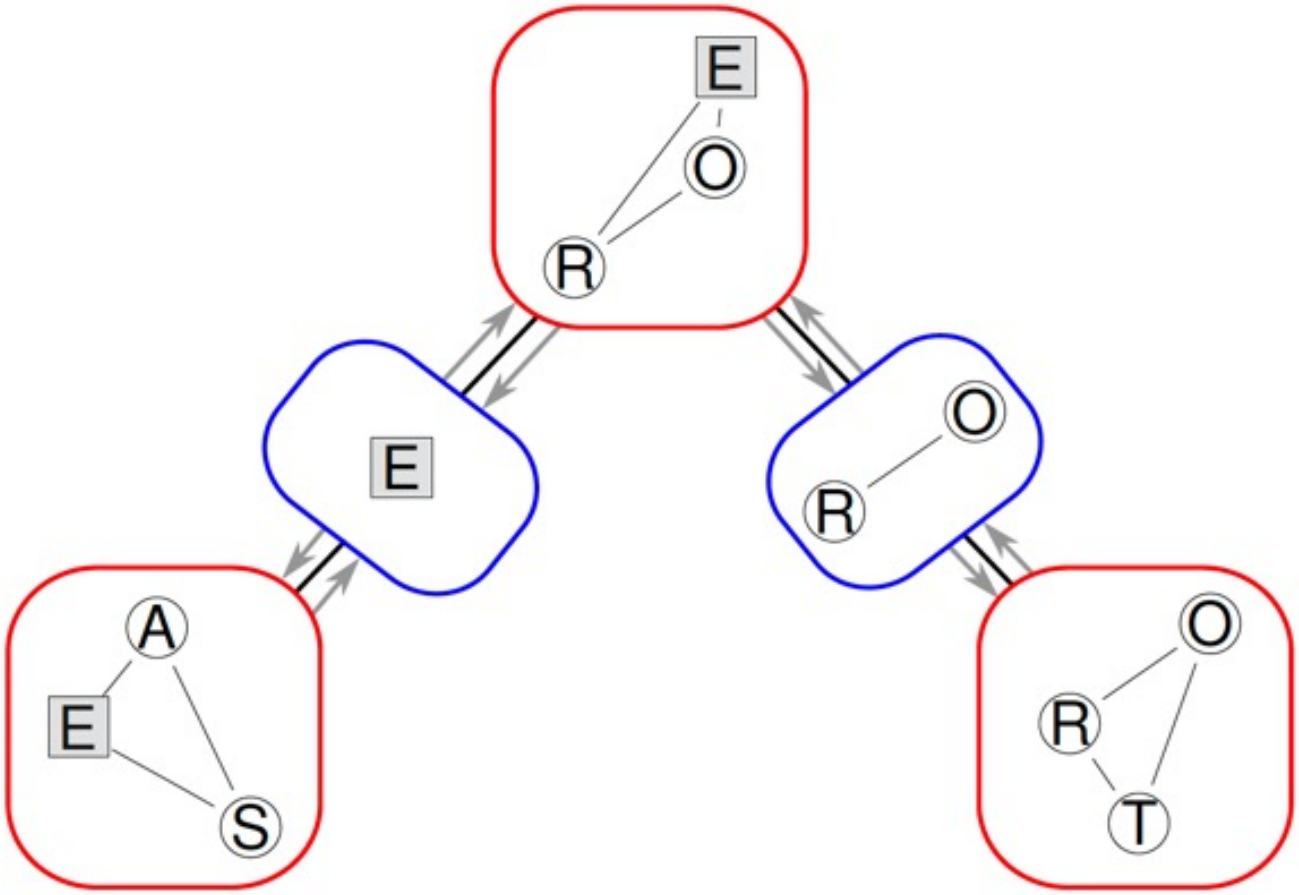
$$\Theta_{C_1} = \mathrm{P}(A, E, S) = \mathrm{P}(A)\,\mathrm{P}(S)\,\mathrm{P}(E \mid A, S)$$
$$\Theta_{C_2} = \mathrm{P}(E, O, R) = \mathrm{P}(O \mid E)\,\mathrm{P}(R \mid E)\,\mathrm{P}(E)$$
$$\Theta_{C_3} = \mathrm{P}(O, R, T) = \mathrm{P}(T \mid O, R)\,\mathrm{P}(O), \mathrm{P}(R)$$

and those for the separators are:

$$\Theta_{S_{12}} = \mathrm{P}(E)$$
$$\Theta_{S_{23}} = \mathrm{P}(O, R)$$

All can be readily computed from the local distributions in the BN.

Back to our Train Example – propagation



Say we set Education to "high school"; we can change it directly in S_{12}, but then we need to propagate the changes to C_1 and C_2; and from C_2 to S_{23} and to C_3. This procedure is called belief propagation by message passing.

Back to our Train Example - queries

Junction trees and belief propagation as implemented in the **gRain** package. Suppose we would like to investigate the distribution of Sex and Travel given the evidence that Education is "high school".

First, we convert the BN from **bnlearn** to its equivalent in **gRain** with `as.grain()` and we construct the junction tree with `compile()`.

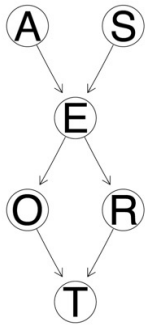
```
library(gRain)
junction = compile(as.grain(bn))
```

Then we set the evidence on the node, fixing it to "high school" with probability 1 with `setEvidence()`.

```
jedu = setEvidence(junction, nodes = "E", states = "high")
```

And after that, we can perform our conditional probability query with `querygrain()`, which also takes care of the belief propagation.

```
SxT.cpt = querygrain(jedu, nodes = c("S", "T"),
                     type = "joint")
```



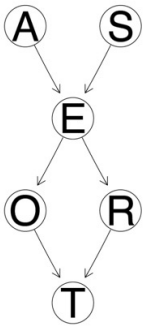
Back to our Train Example - queries

The result of our query is the joint distribution of Sex and Travel given that Education is "high school".

```
SxT.cpt
##    T
## S     car train  other
##   M 0.343 0.174 0.0962
##   F 0.217 0.110 0.0609
```

Similarly, we can use `querygrain()` compute the marginal distributions of Sex and Travel conditional on Education.

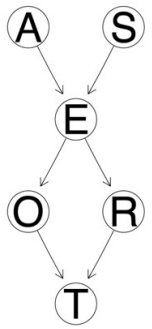
```
querygrain(jedu, nodes = c("S", "T"), type = "marginal")
## $S
## S
##     M     F
## 0.613 0.387
##
## $T
## T
##   car train other
## 0.559 0.283 0.157
```



Back to our Train Example - queries

Interestingly, we can also compute the conditional distribution of Travel given Sex (still conditioning on Education being "high school"), which turns out to be:

```
querygrain(jedu, nodes = c("S", "T"), type = "conditional")
##        S
## T           M     F
##   car   0.613 0.387
##   train 0.613 0.387
##   other 0.613 0.387
```



This makes sense in the light of d-separation, which implies conditional independence.

```
dsep(bn, x = "S", y = "T", z = "E")
## [1] TRUE
```

Logical Sampling

1. Order the variables in $\mathbf{X}$ according to the topological partial ordering implied by G, say $X_{(1)} \prec X_{(2)} \prec \ldots \prec X_{(N)}$.

2. Set $n_{\mathbf{E}} = 0$ and $n_{\mathbf{E},\mathbf{q}} = 0$.

3. For a suitably large number of samples $\mathbf{x} = (x_1, \ldots, x_N)$:

   3.1 generate $x_{(i)}, i = 1, \ldots, N$ from $X_{(i)} \mid \Pi_{X_{(i)}}$ taking advantage of the fact that, thanks to the topological ordering, by the time we are considering X_i we have already generated the values of all its parents $\Pi_{X_{(i)}}$;

   3.2 if $\mathbf{x}$ includes $\mathbf{E}$, set $n_{\mathbf{E}} = n_{\mathbf{E}} + 1$;

   3.3 if $\mathbf{x}$ includes both $\mathbf{Q} = \mathbf{q}$ and $\mathbf{E}$, set $n_{\mathbf{E},\mathbf{q}} = n_{\mathbf{E},\mathbf{q}} + 1$.

4. Estimate $\mathrm{P}(\mathbf{Q} \mid \mathbf{E}, G, \Theta)$ with $n_{\mathbf{E},\mathbf{q}}/n_{\mathbf{E}}$.

Logical Sampling

First, we sample the particles from the BN with `rbn()`, which takes a `bn.fit` object and the number of random samples to generate as arguments.

```
particles = rbn(bn, 10^6)
head(particles, n = 5)
```

```
##       A    E   O   R S     T
## 1   old high emp big M train
## 2   old high emp big M   car
## 3 adult high emp big F   car
## 4   old high emp big M other
## 5 young high emp big M   car
```

The particles are have the correct types and format as derived from the BN, and they are stored in a data frame that has the same structure as that of the data that were used to learn the BN (if any).

Logical Sampling

Then we count how many of those samples that match the evidence $\mathbf{E}$ to estimate $\mathrm{P}(\mathbf{E})$.

```
partE = particles[(particles[, "E"] == "high"), ]
nE = nrow(partE)
```

We also count how many of those samples that match the evidence $\mathbf{E}$ and the query $\mathbf{Q} = \mathbf{q}$ to estimate $\mathrm{P}(\mathbf{Q} = \mathbf{q}, \mathbf{E})$.

```
partEq =
  partE[(partE[, "S"] == "M") & (partE[, "T"] == "car"), ]
nEq = nrow(partEq)
```

Finally, we estimate

$$\mathrm{P}(\mathbf{Q} = \mathbf{q} \mid \mathbf{E}) = \frac{\mathrm{P}(\mathbf{Q} = \mathbf{q}, \mathbf{E})}{\mathrm{P}(\mathbf{E})}.$$

```
nEq/nE
## [1] 0.343
```

Logical Sampling

Then we count how many of those samples that match the evidence $\mathbf{E}$ to estimate $\mathrm{P}(\mathbf{E})$.

```
partE = particles[(particles[, "E"] == "high"), ]
nE = nrow(partE)
```

We also count how many of those samples that match the evidence $\mathbf{E}$ and the query $\mathbf{Q} = \mathbf{q}$ to estimate $\mathrm{P}(\mathbf{Q} = \mathbf{q}, \mathbf{E})$.

```
partEq =
  partE[(partE[, "S"] == "M") & (partE[, "T"] == "car"), ]
nEq = nrow(partEq)
```

Finally, we estimate

$$\mathrm{P}(\mathbf{Q} = \mathbf{q} \mid \mathbf{E}) = \frac{\mathrm{P}(\mathbf{Q} = \mathbf{q}, \mathbf{E})}{\mathrm{P}(\mathbf{E})}.$$

Limitation: A small proportion may have the evidence present!

```
nEq/nE
## [1] 0.343
```

Logical Sampling

These steps are implemented in `cpquery()`, with the obvious arguments:

- `event` is $\mathbf{Q}$;
- `evidence` is $\mathbf{E}$;
- `method` is `"ls"` for logic sampling (the default);
- `n` is the number of particles.

```
cpquery(bn, event = (S == "M") & (T == "car"),
          evidence = (E == "high"), method = "ls", n = 10^6)
## [1] 0.343
```

Both `event` end `evidence` are expressions that are evaluated on the particles much like `subset()` would, so they must evaluate to a vector of `TRUE` and `FALSE` values (hence `&` and not `&&`).

Logical Sampling

More Complex Queries….

Specifying the arguments requires some care, but the result is an extremely flexible framework to compute the probability of arbitrary combinations of events.

As an example of a more complex query, we can compute

$$\mathrm{P}(\mathtt{S} = \mathtt{M}, \mathtt{T} = \mathtt{car} \mid \{\mathtt{A} = \mathtt{young}, \mathtt{E} = \mathtt{uni}\} \cup \{\mathtt{A} = \mathtt{adult}\}),$$

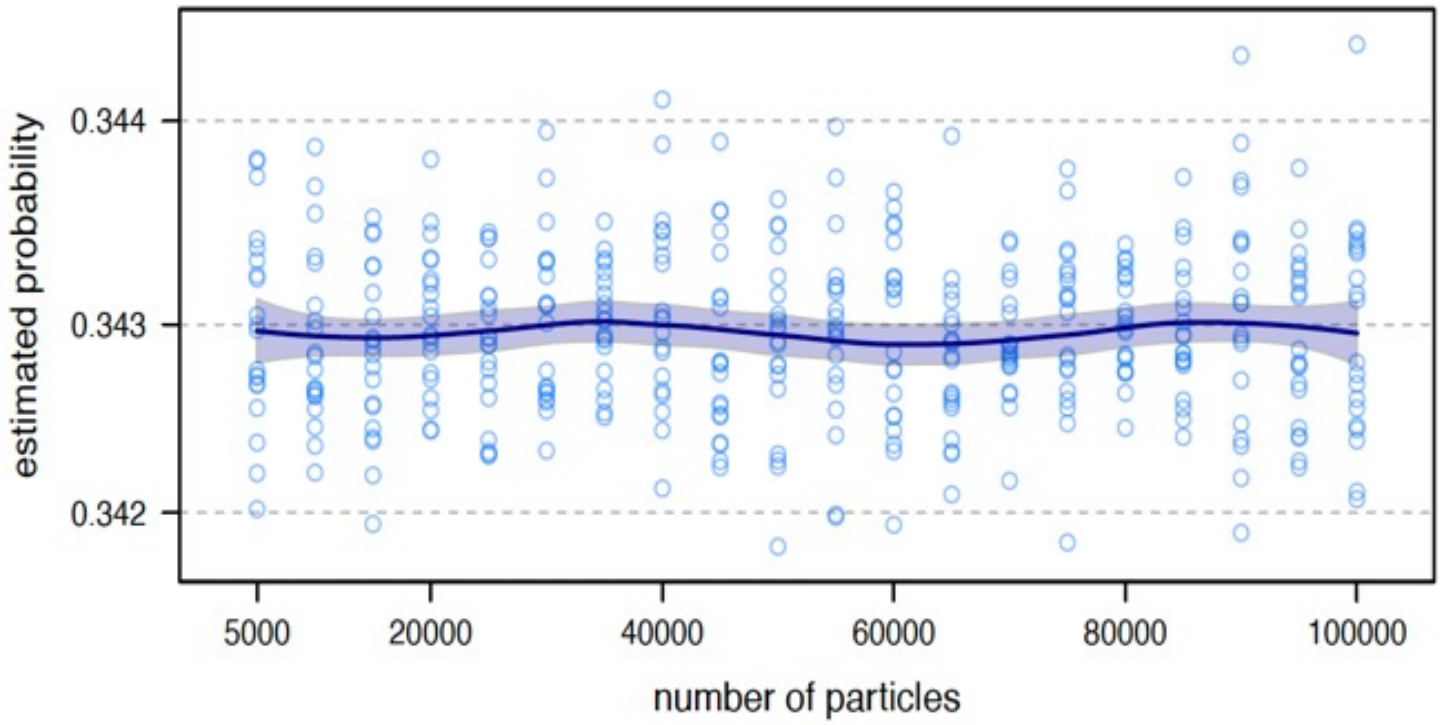
the probability of a man travelling by car given that his Age is young and his Education is uni or that he is an adult, regardless of his Education. That would be:

```
cpquery(bn, event = (S == "M") & (T == "car"),
  evidence = ((A == "young") & (E == "uni")) | (A == "adult"))
## [1] 0.338
```

Logical Sampling

Sample Sizes ….

```
nparticles = seq(from = 5 * 10^3, to = 10^5, by = 5 * 10^3)
prob = matrix(0, nrow = length(nparticles), ncol = 20)
for (i in seq_along(nparticles))
  for (j in 1:20)
    prob[i, j] = cpquery(bn, event = (S == "M") & (T == "car"),
                         evidence = (E == "high"), method = "ls", n = 10^6)
```



Likelihood Weighting Algorithm

An improvement over logic sampling, designed to solve this problem, is a form of importance sampling called likelihood weighting. Unlike logic sampling, all the particles generated by likelihood weighting include the evidence $\mathbf{E}$ by design.

---

1. Order the variables in $\mathbf{X}$ according to the topological ordering implied by G, say $X_{(1)} \prec X_{(2)} \prec \ldots \prec X_{(N)}$.
2. Set $w_{\mathbf{E}} = 0$ and $w_{\mathbf{E},\mathbf{q}} = 0$.
3. For a suitably large number of samples $\mathbf{x} = (x_1, \ldots, x_N)$:
   - 3.1 generate $x_{(i)}, i = 1, \ldots, N$ from $X_{(i)} \mid \Pi_{X_{(i)}}$ using the values $e_1, \ldots, e_k$ specified by the hard evidence $\mathbf{E}$ for $X_{i_1}, \ldots, X_{i_k}$.
   - 3.2 compute the weight $w_{\mathbf{x}} = \prod \mathrm{P}(X_{i^*} = e_* \mid \Pi_{X_{i^*}})$
   - 3.3 set $w_{\mathbf{E}} = w_{\mathbf{E}} + w_{\mathbf{x}}$;
   - 3.4 if $\mathbf{x}$ includes $\mathbf{Q} = \mathbf{q}$, set $w_{\mathbf{E},\mathbf{q}} = w_{\mathbf{E},\mathbf{q}} + w_{\mathbf{x}}$.
4. Estimate $\mathrm{P}(\mathbf{Q} \mid \mathbf{E}, G, \Theta)$ with $w_{\mathbf{E},\mathbf{q}}/w_{\mathbf{E}}$.

Likelihood Weighting Algorithm

We do not want to sample from the original BN, but from the BN in which all the nodes $X_{i_1}, \ldots, X_{i_k}$ in $\mathbf{E}$ are fixed. This network is called the mutilated network.

```
mutbn = mutilated(bn, list(E = "high"))
coef(mutbn$E)
```

```
## high  uni
##    1    0
```

Simply sampling from `mutbn` is not a valid approach. If we do so, the probability we obtain is $\mathrm{P}(\mathbf{Q}, \mathbf{E} \mid G, \Theta)$, not $\mathrm{P}(\mathbf{Q} \mid \mathbf{E}, G, \Theta)$!

Firstly, we sample particles from the original BN one more time.

```
particles = rbn(mutbn, 10^6)
partQ = particles[(particles[, "S"] == "M") &
                  (particles[, "T"] == "car"), ]
```

Likelihood Weighting Algorithm

A simple empirical checks tells us that the naive estimate we could draw from `mutbn` is wrong, since it does not match the exact value we got earlier.

```
nrow(partQ) / nrow(particles)
## [1] 0.336
```

The weights adjust for the fact that we are sampling from the mutilated BN instead of original BN. The weights are just the likelihood components for the particles associated with the nodes we are conditioning on (E in this case).

```
w = logLik(bn, particles, nodes = "E", by.sample = TRUE)
wEq = sum(exp(w[(particles[, "S"] == "M") &
                (particles[, "T"] == "car")]))
wE = sum(exp(w))
wEq/wE
## [1] 0.343
```

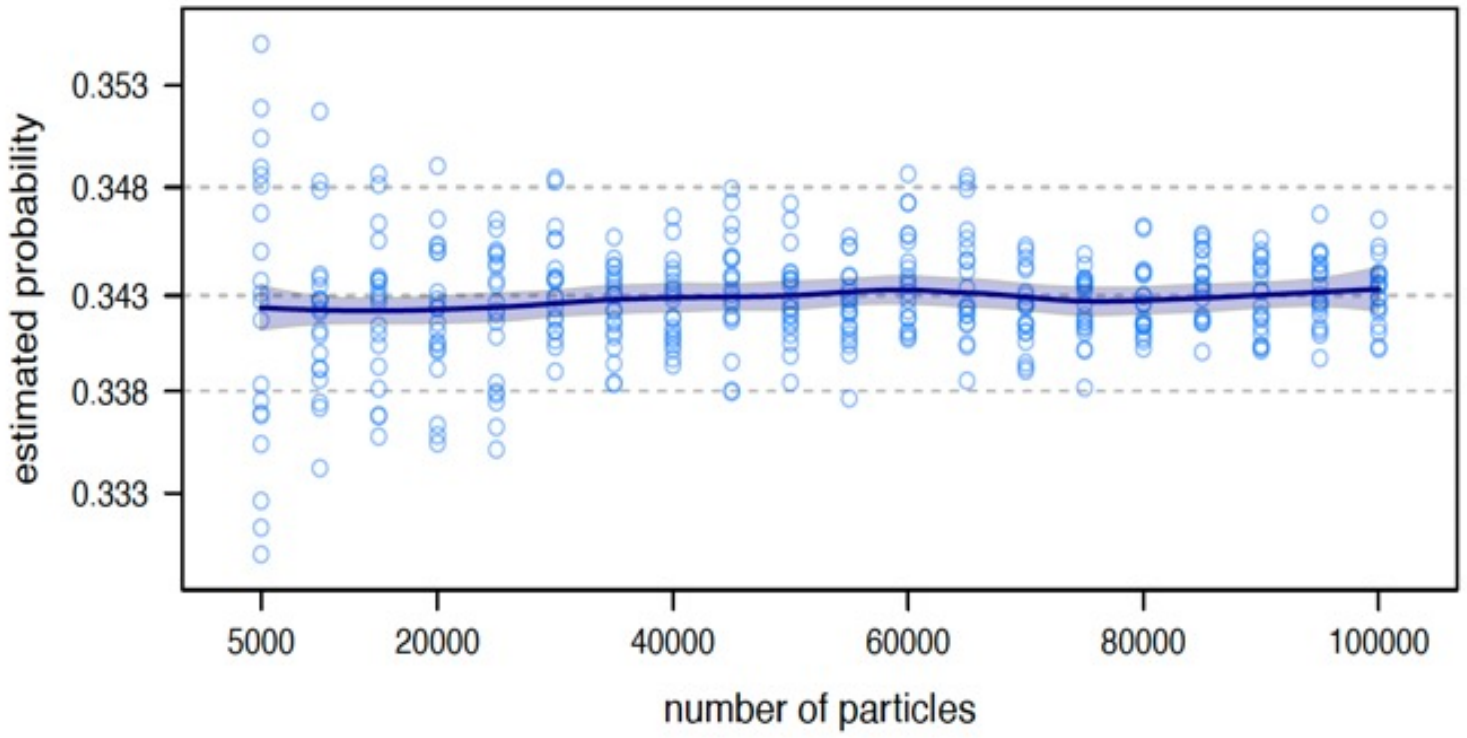
Likelihood Weighting Algorithm

More conveniently, we can perform likelihood weighting with `cpquery` by setting `method = "lw"` and specifying the evidence as a named list with one element for each node we are conditioning on.

```
cpquery(bn, event = (S == "M") & (T == "car"),
          evidence = list(E = "high"), method = "lw", n = 5 * 10^4)
## [1] 0.343
```

Likelihood Weighting Algorithm

```
nparticles = seq(from = 5 * 10^3, to = 10^5, by = 5 * 10^3)
prob = matrix(0, nrow = length(nparticles), ncol = 20)
for (i in seq_along(nparticles))
  for (j in 1:20)
    prob[i, j] = cpquery(bn, event = (S == "M") & (T == "car"),
                         evidence = list(E = "high"), method = "lw",
                         n = nparticles[i])
```



Why Likelihood Weighting Algorithm ?

Logic sampling will be computationally inefficient and very inaccurate if $\mathrm{P}(\mathbf{E})$ is small because most particles will be discarded without contributing to the estimation of $\mathrm{P}(\mathbf{Q} \mid \mathbf{E})$.

```
extreme.dag = model2network("[A][B|A]")
A.prob = array(c(0.999999, 0.000001), dim = 2,
           dimnames = list(A = c("a1", "a2")))
B.prob = array(c(0.5, 0.5, 0.75, 0.25), dim = c(2, 2),
           dimnames = list(B = c("b1", "b2"), A = c("a1", "a2")))
extreme.bn = custom.fit(extreme.dag, list(A = A.prob, B = B.prob))
cpquery(extreme.bn, event = (B == "b2"), evidence = (A == "a2"),
  method = "ls", n = 10^6)
```

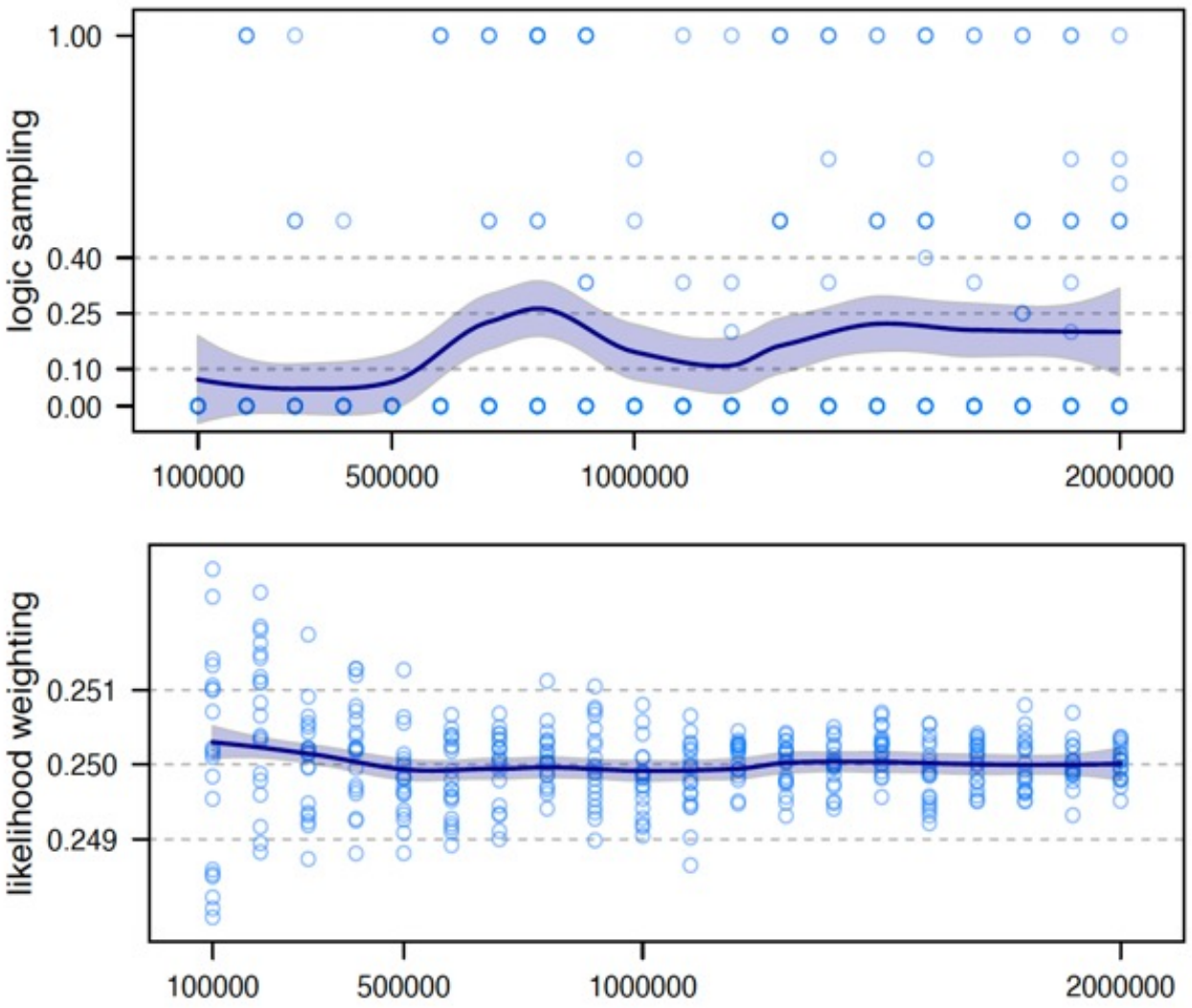
```
## [1] 0
```

This simply does not happen with likelihood weighting.

```
cpquery(extreme.bn, event = (B == "b2"), evidence = list(A = "a2"),
  method = "lw", n = 5 * 10^3)
```

```
## [1] 0.243
```

Why Likelihood Weighting Algorithm ?



Summary

- There are two kinds of questions: conditional probability queries and maximum a posteriori queries. The latter can be answered from the former.
- There are two kinds of way of answering such questions: exact and approximate inference. One uses Bayes theorem and is more accurate, the other Monte Carlo sampling and is more scalable.
- Now we know why diagnostic and prognostic models are interchangeable for inference: they have the same moral graph.